



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра инструментального и прикладного программного обеспечения

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Проектирование и разработка клиент-серверных приложений»

Практическая работа № 1

Многопоточность и параллельные вычисления в Java

Студент группы *ИКБО-11-23, Дицель Н.С.*

(подпись)

Проверил *Преподаватель,
Заринов Евгений Андреевич*

(подпись)

Отчет представлен *«13» сентября 2026 г.*

Москва 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Обработка массива последовательно, многопоточно и с помощью ForkJoin	4
1.1 Перечисление Operation	4
1.2 Последовательная обработка	5
1.3 Многопоточная обработка с помощью Thread	6
1.4 Обработка с помощью ForkJoinPool	7
1.5 Измерение времени выполнения и расхода памяти	8
1.6 Результаты выполнения	11
2 Асинхронная обработка запросов с помощью Future	13
2.1 Программная реализация	13
2.2 Результаты выполнения	17
3 Многопоточная система обработки файлов	18
3.1 Модель данных	18
3.2 Потокбезопасная очередь FileQueue	19
3.3 Генератор и обработчики файлов	21
3.4 Журнал событий и главный класс	23
3.5 Результаты выполнения	24
ЗАКЛЮЧЕНИЕ	27
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	28

ВВЕДЕНИЕ

Современные процессоры многоядерные, поэтому ускорить программу можно, выполняя части работы одновременно. Многопоточность нужна и там, где программа ждёт внешних событий: без неё ожидание блокирует всю программу. Для этого в Java есть пакет `java.util.concurrent`: пулы потоков, интерфейс `Future`, фреймворк `Fork/Join`, явные блокировки и потокобезопасные коллекции [1, 2].

Цель работы – развить навыки работы с многопоточностью и параллельными вычислениями в Java.

Задачи работы:

- а) реализовать обработку массива последовательно и параллельно (`Thread`, `ForkJoin`) и сравнить способы по времени выполнения и расходу памяти;
- б) создать асинхронную программу с использованием `Future`;
- в) спроектировать многопоточную систему обработки файлов с учётом потокобезопасности.

Разделы 1–3 отчёта соответствуют трём заданиям работы. Отчёт оформлен по ГОСТ 7.32–2017 [3].

1 Обработка массива последовательно, многопоточно и с помощью ForkJoin

Требуется найти сумму, максимум и минимум массива из 10 000 целых чисел тремя способами: последовательно, с помощью потоков Thread и с помощью ForkJoin. Обработка каждого элемента сопровождается задержкой 1 мс.

1.1 Перечисление Operation

Чтобы не дублировать параллельные алгоритмы для каждой операции, операция вынесена в перечисление Operation (рисунки 1 и 2). Каждая операция задаётся нейтральным элементом identity() и ассоциативной функцией combine(). Благодаря ассоциативности массив можно разбить на части, обработать их в разных потоках и объединить результаты в любом порядке. Результат хранится в long, так как сумма 10 000 значений int может выйти за пределы int.

```
1 package task1;
2
3 /**
4  * Операция свёртки массива. Каждая операция задаётся нейтральным элементом
5  * и ассоциативной функцией объединения, поэтому частичные результаты,
6  * посчитанные в разных потоках, можно безопасно объединять.
7  */
8 public enum Operation {
9     SUM("Сумма элементов") {
10         @Override
11         public long identity() {
12             return 0L;
13         }
14
15         @Override
16         public long combine(long a, long b) {
17             return a + b;
18         }
19     },
20     MAX("Максимальный элемент") {
21         @Override
22         public long identity() {
23             return Long.MIN_VALUE;
24         }
25
26         @Override
27         public long combine(long a, long b) {
28             return Math.max(a, b);
29         }
30     },
```

Рисунок 1 – Перечисление Operation, операции SUM и MAX

```

31 MIN("Минимальный элемент") {
32     @Override
33     public long identity() {
34         return Long.MAX_VALUE;
35     }
36
37     @Override
38     public long combine(long a, long b) {
39         return Math.min(a, b);
40     }
41 };
42
43 private final String title;
44
45 Operation(String title) {
46     this.title = title;
47 }
48
49 public String getTitle() {
50     return title;
51 }
52
53 public abstract long identity();
54
55 public abstract long combine(long a, long b);
56 }

```

Рисунок 2 – Перечисление Operation, операция MIN и общие методы

1.2 Последовательная обработка

Все три способа используют общий метод `processRange()` класса `ArrayProcessor` (рисунок 3). Он обрабатывает отрезок `[from, to)`: для каждого элемента выполняет задержку `delay()` и объединяет элемент с аккумулятором. Последовательный вариант `sequential()` вызывает `processRange()` для всего массива, поэтому способы различаются только разбиением массива и выбором потоков.

```

1 package task1;
2
3 import java.util.concurrent.ForkJoinPool;
4 import java.util.concurrent.RecursiveTask;
5
6 /**
7  * Три реализации одной и той же операции над массивом:
8  * последовательная, на потоках Thread и на ForkJoinPool.
9  * Обработка каждого элемента сопровождается задержкой {@link #DELAY_MS} мс.
10 */
11 public final class ArrayProcessor {
12
13     public static final long DELAY_MS = 1;
14
15     private ArrayProcessor() {
16     }
17
18     /** Последовательная обработка отрезка [from, to). Используется всеми реализациями.
19     */
20     static long processRange(int[] array, int from, int to, Operation op) {
21         long acc = op.identity();
22         for (int i = from; i < to; i++) {
23             delay();
24             acc = op.combine(acc, array[i]);
25         }
26         return acc;
27     }
28
29     public static long sequential(int[] array, Operation op) {
30         return processRange(array, 0, array.length, op);
31     }
32 }

```

Рисунок 3 – Класс ArrayProcessor, общий метод processRange() и последовательная обработка

1.3 Многопоточная обработка с помощью Thread

В методе multithreaded() (рисунок 4) массив делится на threadCount частей размером $chunk = (n + threadCount - 1) / threadCount$, и каждая часть обрабатывается отдельным потоком Thread [4]. Основной поток дожидается рабочих потоков через join() и объединяет их результаты. Синхронизация не нужна: массив только читается, а каждый поток пишет результат в свою ячейку results[index].

```

37     public static long multithreaded(final int[] array, final Operation op, int
    »         threadCount)
38         throws InterruptedException {
39         final long[] results = new long[threadCount];
40         Thread[] workers = new Thread[threadCount];
41         int chunk = (array.length + threadCount - 1) / threadCount;
42
43         for (int t = 0; t < threadCount; t++) {
44             final int index = t;
45             final int from = Math.min(array.length, t * chunk);
46             final int to = Math.min(array.length, from + chunk);
47             workers[t] = new Thread(new Runnable() {
48                 @Override
49                 public void run() {
50                     results[index] = processRange(array, from, to, op);
51                 }
52             }, "worker-" + t);
53             workers[t].start();
54         }
55
56         long acc = op.identity();
57         for (int t = 0; t < threadCount; t++) {
58             workers[t].join();
59             acc = op.combine(acc, results[t]);
60         }
61         return acc;
62     }
63
64     public static long forkJoin(int[] array, Operation op, ForkJoinPool pool) {
65         return pool.invoke(new RangeTask(array, 0, array.length, op));
66     }

```

Рисунок 4 – Класс ArrayProcessor, многопоточная обработка и запуск ForkJoin

1.4 Обработка с помощью ForkJoinPool

Метод `forkJoin()` (рисунок 4, строки 64–66) передаёт корневую задачу `RangeTask` в пул `ForkJoinPool` методом `invoke()` [5]. Задача `RangeTask` (рисунок 5) обрабатывает отрезок не длиннее 250 элементов напрямую, а более длинный делит пополам: левую половину отправляет в очередь пула через `fork()`, правую считает в текущем потоке через `compute()` и объединяет результаты после `join()`. Свободные потоки пула забирают подзадачи из очередей занятых потоков (work stealing).

```

69     static final class RangeTask extends RecursiveTask<Long> {
70         private static final long serialVersionUID = 1L;
71         private static final int THRESHOLD = 250;
72
73         private final int[] array;
74         private final int from;
75         private final int to;
76         private final Operation op;
77
78         RangeTask(int[] array, int from, int to, Operation op) {
79             this.array = array;
80             this.from = from;
81             this.to = to;
82             this.op = op;
83         }
84
85         @Override
86         protected Long compute() {
87             if (to - from <= THRESHOLD) {
88                 return processRange(array, from, to, op);
89             }
90             int mid = (from + to) >> 1;
91             RangeTask left = new RangeTask(array, from, mid, op);
92             RangeTask right = new RangeTask(array, mid, to, op);
93             left.fork();
94             long rightResult = right.compute();
95             long leftResult = left.join();
96             return op.combine(leftResult, rightResult);
97         }
98     }
99
100     private static void delay() {
101         try {
102             Thread.sleep(DELAY_MS);
103         } catch (InterruptedException e) {
104             Thread.currentThread().interrupt();
105             throw new IllegalStateException("Обработка прервана", e);
106         }
107     }
108 }

```

Рисунок 5 – Класс ArrayProcessor, рекурсивная задача RangeTask и метод delay()

1.5 Измерение времени выполнения и расхода памяти

Главный класс задания приведён на рисунках 6–8. Программа заполняет массив случайными числами из [0; 1000) и создаёт пул ForkJoinPool с числом потоков, равным числу логических ядер (рисунок 6).

```

1 package task1;
2
3 import java.util.Random;
4 import java.util.concurrent.Callable;
5 import java.util.concurrent.ForkJoinPool;
6
7 /**
8  * Задание 1. Сравнение последовательной, многопоточной и ForkJoin обработки
9  * массива из 10 000 элементов по времени выполнения и расходу памяти.
10  *
11  * Запуск: java task1.Main [SUM/MAX/MIN] [размер массива]
12  */
13 public class Main {
14
15     public static void main(String[] args) throws Exception {
16         final Operation op = args.length > 0 ? Operation.valueOf(args[0].toUpperCase())
17             : Operation.SUM;
18         int size = args.length > 1 ? Integer.parseInt(args[1]) : 10_000;
19         final int threads = Runtime.getRuntime().availableProcessors();
20
21         final int[] array = new int[size];
22         Random random = new Random();
23         for (int i = 0; i < size; i++) {
24             array[i] = random.nextInt(1000);
25         }
26
27         System.out.println("Операция: " + op.getTitle());
28         System.out.println("Размер массива: " + size + ", задержка на элемент: " +
29             »      ArrayProcessor.DELAY_MS + " мс");
30         System.out.println("Доступно ядер: " + threads);
31         System.out.println();
32
33         final ForkJoinPool pool = new ForkJoinPool(threads);

```

Рисунок 6 – Класс task1.Main, подготовка данных

Каждый способ передаётся в метод `measure()` (рисунок 7), после чего программа выводит сводную таблицу, проверяет совпадение результатов и вычисляет ускорение относительно последовательного варианта.

```

33     Result sequential = measure("Последовательно", new Callable<Long>() {
34         @Override
35         public Long call() {
36             return ArrayProcessor.sequential(array, op);
37         }
38     });
39     Result multithreaded = measure("Многопоточно (" + threads + " Thread)", new
    »     Callable<Long>() {
40         @Override
41         public Long call() throws Exception {
42             return ArrayProcessor.multithreaded(array, op, threads);
43         }
44     });
45     Result forkJoin = measure("ForkJoinPool (" + threads + ")", new Callable<Long>()
    »     {
46         @Override
47         public Long call() {
48             return ArrayProcessor.forkJoin(array, op, pool);
49         }
50     });
51     pool.shutdown();
52
53     System.out.println();
54     System.out.printf("%-28s %15s %12s %14s%n", "Способ", "Результат", "Время, мс",
    »         "Память, КБ");
55     for (Result r : new Result[]{sequential, multithreaded, forkJoin}) {
56         System.out.printf("%-28s %15d %12d %14d%n", r.name, r.value, r.timeMs,
    »             r.memoryKb);
57     }
58     System.out.println();
59
60     boolean consistent = sequential.value == multithreaded.value && sequential.value
    »         == forkJoin.value;
61     System.out.println("Результаты совпадают: " + (consistent ? "да" : "НЕТ"));
62     System.out.printf("Ускорение Thread: %.2fx, ForkJoin: %.2fx%n",
63         (double) sequential.timeMs / multithreaded.timeMs,
64         (double) sequential.timeMs / forkJoin.timeMs);
65 }

```

Рисунок 7 – Класс task1.Main, запуск замеров и вывод результатов

Метод `measure()` (рисунок 8) засекает время через `System.nanoTime()`, а расход памяти оценивает по приросту занятой кучи `totalMemory()` – `freeMemory()` после вызова `System.gc()`. Такая оценка приближённая: она не учитывает стеки потоков и включает служебные объекты виртуальной машины.

```

67     private static Result measure(String name, Callable<Long> action) throws Exception {
68         System.gc();
69         long memoryBefore = usedMemory();
70         long start = System.nanoTime();
71
72         long value = action.call();
73
74         long timeMs = (System.nanoTime() - start) / 1_000_000;
75         long memoryKb = Math.max(0, usedMemory() - memoryBefore) / 1024;
76         System.out.println(name + ": готово за " + timeMs + " мс");
77         return new Result(name, value, timeMs, memoryKb);
78     }
79
80     private static long usedMemory() {
81         Runtime runtime = Runtime.getRuntime();
82         return runtime.totalMemory() - runtime.freeMemory();
83     }
84
85     private static final class Result {
86         final String name;
87         final long value;
88         final long timeMs;
89         final long memoryKb;
90
91         Result(String name, long value, long timeMs, long memoryKb) {
92             this.name = name;
93             this.value = value;
94             this.timeMs = timeMs;
95             this.memoryKb = memoryKb;
96         }
97     }

```

Рисунок 8 – Класс task1.Main, методы measure(), usedMemory() и класс Result

1.6 Результаты выполнения

Программа запускалась для операций SUM, MAX и MIN

```

C:\Users\relax\Documents\PiRKSP\Практическая работа №1>run.bat 1 SUM 10000
Операция: Сумма элементов
Размер массива: 10000, задержка на элемент: 1 мс
Доступно ядер: 16

Последовательно: готово за 17543 мс
Многопоточно (16 Thread): готово за 1048 мс
ForkJoinPool (16): готово за 1373 мс

Способ                                Результат      Время, мс      Память, КБ
Последовательно                       4989048        17543         1351
Многопоточно (16 Thread)               4989048        1048          82
ForkJoinPool (16)                     4989048        1373          287

Результаты совпадают: да
Ускорение Thread: 16,74x, ForkJoin: 12,78x

```

Рисунок 9 – Результат выполнения задания 1 для операции SUM

```

C:\Users\relax\Documents\PiRKSP\Практическая работа №1>run.bat 1 MAX 10000
Операция: Максимальный элемент
Размер массива: 10000, задержка на элемент: 1 мс
Доступно ядер: 16

Последовательно: готово за 18001 мс
Многопоточно (16 Thread): готово за 1106 мс
ForkJoinPool (16): готово за 1208 мс

Способ                Результат    Время, мс    Память, КБ
Последовательно                999        18001        1351
Многопоточно (16 Thread)        999         1106         82
ForkJoinPool (16)                999         1208         396

Результаты совпадают: да
Ускорение Thread: 16,28x, ForkJoin: 14,90x

```

Рисунок 10 – Результат выполнения задания 1 для операции MAX

```

C:\Users\relax\Documents\PiRKSP\Практическая работа №1>run.bat 1 MIN 10000
Операция: Минимальный элемент
Размер массива: 10000, задержка на элемент: 1 мс
Доступно ядер: 16

Последовательно: готово за 18039 мс
Многопоточно (16 Thread): готово за 1039 мс
ForkJoinPool (16): готово за 1338 мс

Способ                Результат    Время, мс    Память, КБ
Последовательно                0        18039        1351
Многопоточно (16 Thread)        0         1039         82
ForkJoinPool (16)                0         1338        256

Результаты совпадают: да
Ускорение Thread: 17,36x, ForkJoin: 13,48x

```

Рисунок 11 – Результат выполнения задания 1 для операции MIN

Во всех запусках результаты трёх способов совпали. Последовательная обработка заняла 17,5–18,0 с, многопоточная – 1039–1106 мс, ForkJoin – 1208–1373 мс. Ускорение составило 16,3–17,4 раза для Thread и 12,8–14,9 раза для ForkJoin, то есть близко к числу потоков (16). Так получилось потому, что обработка элемента сводится к ожиданию: спящие потоки не занимают ядра, и почти вся работа выполняется параллельно [6]. Последовательный вариант выполнялся дольше ожидаемых 10 с, потому что в Windows sleep(1) длится около 1,75 мс. ForkJoin немного медленнее Thread: подзадачи распределяются между потоками постепенно, а создание задач и работа с очередями пула требуют дополнительного времени. Памяти кучи ForkJoin также потребовал больше (256–396 КБ против 82 КБ у Thread) – на объекты задач и очереди пула.

2 Асинхронная обработка запросов с помощью Future

Пользователь вводит число, программа возводит его в квадрат с задержкой 1–5 с, используя Future. Новые числа можно вводить, не дожидаясь результатов предыдущих запросов.

2.1 Программная реализация

Решение построено на трёх видах потоков: основной поток читает ввод, потоки пула ExecutorService [7] вычисляют квадраты, а отдельный поток выводит готовые результаты. Импорт и объявление класса приведены на рисунке 12.

```
1 package task2;
2
3 import java.util.List;
4 import java.util.Random;
5 import java.util.Scanner;
6 import java.util.concurrent.Callable;
7 import java.util.concurrent.CopyOnWriteArrayList;
8 import java.util.concurrent.ExecutionException;
9 import java.util.concurrent.ExecutorService;
10 import java.util.concurrent.Executors;
11 import java.util.concurrent.Future;
12 import java.util.concurrent.TimeUnit;
13 import java.util.concurrent.atomic.AtomicInteger;
14
15 /**
16  * Задание 2. Асинхронная обработка запросов пользователя с помощью Future.
17  * Пользователь вводит число, программа возводит его в квадрат с задержкой 1–5 секунд.
18  * Новые числа можно вводить, не дожидаясь результатов предыдущих запросов.
19  */
20 public class Main {
21
22     private static final Random RANDOM = new Random();
```

Рисунок 12 – Класс task2.Main, импорт и объявление класса

В методе main() (рисунок 13) создаются пул Executors.newCachedThreadPool(), потокобезопасный список запросов CopyOnWriteArrayList и счётчик AtomicInteger. Для каждого числа задача передаётся в пул методом submit(), который сразу возвращает Future<Long>, поэтому ввод не блокируется. Некорректная строка отклоняется, команда status показывает число запросов в обработке, а exit закрывает пул и дожидается оставшихся задач.

```

24     public static void main(String[] args) throws InterruptedException {
25         ExecutorService executor = Executors.newCachedThreadPool();
26         List<Request> pending = new CopyOnWriteArrayList<>();
27         AtomicInteger ids = new AtomicInteger();
28         ResultPrinter printer = new ResultPrinter(pending);
29         printer.start();
30
31         System.out.println("Вводите числа (каждое будет возведено в квадрат с задержкой
32             »      1-5 с).");
33         System.out.println("Команды: status - незавершённые запросы, exit - выход.");
34
35         Scanner scanner = new Scanner(System.in);
36         while (scanner.hasNextLine()) {
37             String line = scanner.nextLine().replace((char) 0xFEFF, ' ').trim();
38             if (line.isEmpty()) {
39                 continue;
40             }
41             if (line.equalsIgnoreCase("exit")) {
42                 break;
43             }
44             if (line.equalsIgnoreCase("status")) {
45                 System.out.println("В обработке: " + pending.size());
46                 continue;
47             }
48
49             final long number;
50             try {
51                 number = Long.parseLong(line);
52             } catch (NumberFormatException e) {
53                 System.out.println("Ошибка: '" + line + "' не является целым числом");
54                 continue;
55             }
56
57             Future<Long> future = executor.submit(new SquareTask(number));
58             Request request = new Request(ids.incrementAndGet(), number, future);
59             pending.add(request);
60             System.out.println("Запрос #" + request.id + " (" + number + ") принят, ждём
61                 »      результат...");
62         }
63
64         executor.shutdown();
65         if (!pending.isEmpty()) {
66             System.out.println("Ожидание завершения запросов: " + pending.size());
67         }
68         executor.awaitTermination(1, TimeUnit.MINUTES);
69         printer.finish();
70         System.out.println("Работа завершена.");
71     }

```

Рисунок 13 – Класс task2.Main, метод main()

Задача SquareTask (рисунок 14) реализует Callable<Long>: засыпает на 1–5 с и вычисляет квадрат методом Math.multiplyExact(), который при переполнении long выбрасывает ArithmeticException. Неизменяемый класс Request хранит номер запроса, число, Future и время приёма.

```

72     private static final class SquareTask implements Callable<Long> {
73         private final long number;
74
75         SquareTask(long number) {
76             this.number = number;
77         }
78
79         @Override
80         public Long call() throws InterruptedException {
81             Thread.sleep(1000 + RANDOM.nextInt(4001));
82             return Math.multiplyExact(number, number);
83         }
84     }
85
86     private static final class Request {
87         final int id;
88         final long number;
89         final Future<Long> future;
90         final long submittedAt = System.currentTimeMillis();
91
92         Request(int id, long number, Future<Long> future) {
93             this.id = id;
94             this.number = number;
95             this.future = future;
96         }
97     }

```

Рисунок 14 – Класс task2.Main, задача SquareTask и класс запроса Request

Поток ResultPrinter (рисунки 15 и 16) каждые 100 мс проверяет future.isDone() и выводит готовые результаты. Вызов get() для завершённой задачи не блокирует поток, а исключение из call() приходит как ExecutionException [8]. Флаг running объявлен volatile, так как его меняет основной поток.

```

103     private static final class ResultPrinter extends Thread {
104         private final List<Request> pending;
105         private volatile boolean running = true;
106
107         ResultPrinter(List<Request> pending) {
108             super("result-printer");
109             this.pending = pending;
110             setDaemon(true);
111         }
112
113         @Override
114         public void run() {
115             while (running || !pending.isEmpty()) {
116                 for (Request request : pending) {
117                     if (request.future.isDone()) {
118                         pending.remove(request);
119                         print(request);
120                     }
121                 }
122                 try {
123                     Thread.sleep(100);
124                 } catch (InterruptedException e) {
125                     return;
126                 }
127             }
128         }

```

Рисунок 15 – Класс task2.Main, поток вывода ResultPrinter, метод run()

```

130     private void print(Request request) {
131         long elapsed = System.currentTimeMillis() - request.submittedAt;
132         try {
133             long square = request.future.get();
134             System.out.println("Запрос #" + request.id + ": (" + request.number +
135                 ")^2 = " + square
136                 + " (через " + elapsed + " мс)");
137         } catch (ExecutionException e) {
138             System.out.println("Запрос #" + request.id + ": ошибка вычисления - " +
139                 e.getCause());
140         } catch (InterruptedException e) {
141             Thread.currentThread().interrupt();
142         }
143
144     void finish() throws InterruptedException {
145         running = false;
146         join();
147     }
148 }

```

Рисунок 16 – Класс task2.Main, методы print() и finish() потока ResultPrinter

2.2 Результаты выполнения

Числа вводились подряд с интервалом около 0,6 с, не дожидаясь результатов. Дополнительно были введены строка abc и число 9 999 999 999 999, квадрат которого не помещается в long. Сеанс показан на рисунке 17; ввод пользователя выделен жёлтым.

```
C:\Users\relax\Documents\PiRKSP\Практическая работа №1>run.bat 2
Вводите числа (каждое будет возведено в квадрат с задержкой 1-5 с).
Команды: status - незавершённые запросы, exit - выход.
5
Запрос #1 (5) принят, ждём результат...
12
Запрос #2 (12) принят, ждём результат...
-7
Запрос #3 (-7) принят, ждём результат...
abc
Ошибка: 'abc' не является целым числом
Запрос #2: (12)^2 = 144 (через 1229 мс)
99999999999999
Запрос #4 (99999999999999) принят, ждём результат...
Запрос #1: (5)^2 = 25 (через 2919 мс)
3
Запрос #5 (3) принят, ждём результат...
status
В обработке: 3
Запрос #4: ошибка вычисления - java.lang.ArithmeticException: long overflow
Запрос #3: (-7)^2 = 49 (через 4576 мс)
status
В обработке: 1
exit
Ожидание завершения запросов: 1
Запрос #5: (3)^2 = 9 (через 3881 мс)
Работа завершена.
```

Рисунок 17 – Результат выполнения задания 2

Каждый запрос принимался сразу, а результаты пришли в порядке готовности (#2, #1, #4, #3, #5), а не в порядке ввода: например, запрос #2 завершился раньше #1. Для abc выведена ошибка ввода, для большого числа – ArithmeticException, полученное через Future. После exit программа дождалась последнего запроса и завершилась.

3 Многопоточная система обработки файлов

Генератор создаёт файлы случайного типа (XML, JSON, XLS) и размера (10–100) с задержкой 100–1000 мс и помещает их в очередь вместимостью 5 файлов. Для каждого типа работает свой обработчик, который забирает только файлы своего типа и обрабатывает файл за время, равное размеру $\times 7$ мс.

3.1 Модель данных

Типы файлов заданы перечислением FileType (рисунок 18), файл – классом File (рисунок 19). Все поля File объявлены final, поэтому объект неизменяем и безопасно передаётся между потоками.

```
1 package task3;
2
3 public enum FileType {
4     XML, JSON, XLS
5 }
```

Рисунок 18 – Перечисление FileType

```
1 package task3;
2
3 /** Неизменяемый объект файла, поэтому его можно безопасно передавать между потоками. */
4 public final class File {
5     private final int id;
6     private final FileType type;
7     private final int size;
8
9     public File(int id, FileType type, int size) {
10         this.id = id;
11         this.type = type;
12         this.size = size;
13     }
14
15     public int getId() {
16         return id;
17     }
18
19     public FileType getType() {
20         return type;
21     }
22
23     public int getSize() {
24         return size;
25     }
26
27     @Override
28     public String toString() {
29         return "File#" + id + "[" + type + ", size=" + size + "]";
30     }
31 }
```

Рисунок 19 – Класс File

3.2 Потокобезопасная очередь FileQueue

Стандартная `BlockingQueue` отдаёт первый элемент очереди, а обработчику нужен первый файл своего типа, поэтому очередь реализована самостоятельно (рисунки 20–22). Состояние очереди защищено одной блокировкой `ReentrantLock` с двумя условиями: `notFull` (есть место) и `fileAdded` (добавлен файл или очередь закрыта). Метод `put()` (рисунок 20) ждёт на `notFull`, пока очередь заполнена, добавляет файл и будит всех обработчиков через `signalAll()`: сигнал `signal()` мог бы достаться обработчику другого типа и потеряться.

```
1 package task3;
2
3 import java.util.Iterator;
4 import java.util.LinkedList;
5 import java.util.concurrent.locks.Condition;
6 import java.util.concurrent.locks.ReentrantLock;
7
8 /**
9  * Потокобезопасная ограниченная очередь файлов.
10  * put() блокируется, пока очередь заполнена; take(type) блокируется,
11  * пока в очереди нет файла нужного типа.
12  */
13 public final class FileQueue {
14     private final int capacity;
15     private final LinkedList<File> files = new LinkedList<>();
16     private final ReentrantLock lock = new ReentrantLock();
17     private final Condition notFull = lock.newCondition();
18     private final Condition fileAdded = lock.newCondition();
19     private boolean closed;
20
21     public FileQueue(int capacity) {
22         this.capacity = capacity;
23     }
24
25     public void put(File file) throws InterruptedException {
26         lock.lockInterruptibly();
27         try {
28             if (closed) {
29                 throw new IllegalStateException("Очередь закрыта");
30             }
31             if (files.size() == capacity) {
32                 log.info("очередь заполнена (" + capacity + "), генератор ждёт");
33             }
34             while (files.size() == capacity) {
35                 notFull.await();
36             }
37             files.addLast(file);
38             fileAdded.signalAll();
39         } finally {
40             lock.unlock();
41         }
42     }
```

Рисунок 20 – Класс `FileQueue`, поля и метод `put()`

Метод `take(type)` (рисунок 21) ищет первый файл нужного типа, удаляет его и будит генератор через `notFull.signal()`. Если файла нет, поток ждёт на `fileAdded`, а после пробуждения повторяет поиск в цикле, что защищает от ложных пробуждений. Если очередь закрыта и файлов типа не осталось, метод возвращает `null`. Блокировка снимается в `finally`.

```
49     public File take(FileType type) throws InterruptedException {
50         lock.lockInterruptibly();
51         try {
52             while (true) {
53                 Iterator<File> iterator = files.iterator();
54                 while (iterator.hasNext()) {
55                     File file = iterator.next();
56                     if (file.getType() == type) {
57                         iterator.remove();
58                         notFull.signal();
59                         return file;
60                     }
61                 }
62                 if (closed) {
63                     return null;
64                 }
65                 fileAdded.await();
66             }
67         } finally {
68             lock.unlock();
69         }
70     }
```

Рисунок 21 – Класс `FileQueue`, метод `take()`

Метод `close()` (рисунок 22) помечает очередь закрытой и будит ожидающих обработчиков. Метод `size()` тоже выполняется под блокировкой, так как `LinkedList` не потокобезопасен.

```

72     /** Новых файлов не будет: обработчики доберут оставшиеся и завершатся. */
73     public void close() {
74         lock.lock();
75         try {
76             closed = true;
77             fileAdded.signalAll();
78         } finally {
79             lock.unlock();
80         }
81     }
82
83     public int size() {
84         lock.lock();
85         try {
86             return files.size();
87         } finally {
88             lock.unlock();
89         }
90     }
91 }

```

Рисунок 22 – Класс FileQueue, методы close() и size()

3.3 Генератор и обработчики файлов

Генератор (рисунок 23) делает паузу 100–1000 мс, создаёт файл и кладёт его в очередь; при заполненной очереди он блокируется в put(). Для случайных чисел используется ThreadLocalRandom, не требующий синхронизации.

```

1  package task3;
2
3  import java.util.concurrent.ThreadLocalRandom;
4
5  /** Генерирует файлы случайного типа и размера (10-100) с задержкой 100-1000 мс. */
6  public final class FileGenerator implements Runnable {
7      private final FileQueue queue;
8      private final int fileCount;
9
10     public FileGenerator(FileQueue queue, int fileCount) {
11         this.queue = queue;
12         this.fileCount = fileCount;
13     }
14
15     @Override
16     public void run() {
17         ThreadLocalRandom random = ThreadLocalRandom.current();
18         FileType[] types = FileType.values();
19         try {
20             for (int id = 1; id <= fileCount; id++) {
21                 Thread.sleep(random.nextInt(100, 1001));
22                 File file = new File(id, types[random.nextInt(types.length)],
23                     » random.nextInt(10, 101));
24                 Log.info("создан " + file);
25                 queue.put(file);
26             }
27         } catch (InterruptedException e) {
28             Thread.currentThread().interrupt();
29         }
30         Log.info("генерация завершена");
31     }
32 }

```

Рисунок 23 – Класс FileGenerator

Обработчик (рисунок 24) в цикле забирает файлы своего типа и имитирует обработку вызовом `Thread.sleep(size × 7)`. Получив `null`, он выводит число обработанных файлов и завершается. Счётчик `processed` используется только своим потоком и не требует синхронизации.

```

1 package task3;
2
3 /** Обработывает файлы только своего типа; время обработки = размер * 7 мс. */
4 public final class FileProcessor implements Runnable {
5     private static final int MS_PER_SIZE_UNIT = 7;
6
7     private final FileQueue queue;
8     private final FileType type;
9     private int processed;
10
11     public FileProcessor(FileQueue queue, FileType type) {
12         this.queue = queue;
13         this.type = type;
14     }
15
16     @Override
17     public void run() {
18         try {
19             File file;
20             while ((file = queue.take(type)) != null) {
21                 long time = (long) file.getSize() * MS_PER_SIZE_UNIT;
22                 Log.info("начата обработка " + file + " (" + time + " мс), в очереди: "
23                     » + queue.size());
24                 Thread.sleep(time);
25                 processed++;
26                 Log.info("обработан " + file);
27             }
28         } catch (InterruptedException e) {
29             Thread.currentThread().interrupt();
30         }
31         Log.info("обработчик " + type + " завершён, обработано файлов: " + processed);
32     }
33 }

```

Рисунок 24 – Класс FileProcessor

3.4 Журнал событий и главный класс

Класс Log (рисунок 25) выводит сообщения с временем до миллисекунд и именем потока, что позволяет проследить параллельную работу по журналу.

```

1 package task3;
2
3 import java.time.LocalDateTime;
4 import java.time.format.DateTimeFormatter;
5
6 final class Log {
7     private static final DateTimeFormatter FORMAT =
8         » DateTimeFormatter.ofPattern("HH:mm:ss.SSS");
9
10     private Log() {
11     }
12
13     static void info(String message) {
14         System.out.printf("%s [%-14s] %s%n", LocalDateTime.now().format(FORMAT),
15             » Thread.currentThread().getName(), message);
16     }
17 }

```

Рисунок 25 – Класс Log

Главный класс (рисунок 26) запускает по обработчику на каждый тип файлов и поток генератора. После окончания генерации он закрывает очередь и дожидается обработчиков через `join()`, поэтому все созданные файлы обрабатываются, а потоки завершаются штатно.

```
1 package task3;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 /**
7  * Задание 3. Система обработки файлов: генератор -> очередь (вместимость 5) ->
8  * обработчики по типам.
9  *
10  * Запуск: java task3.Main [количество файлов]
11  */
12 public class Main {
13     private static final int QUEUE_CAPACITY = 5;
14
15     public static void main(String[] args) throws InterruptedException {
16         int fileCount = args.length > 0 ? Integer.parseInt(args[0]) : 20;
17         FileQueue queue = new FileQueue(QUEUE_CAPACITY);
18
19         List<Thread> processors = new ArrayList<>();
20         for (FileType type : FileType.values()) {
21             Thread thread = new Thread(new FileProcessor(queue, type), "processor-" +
22                 type);
23             processors.add(thread);
24             thread.start();
25         }
26
27         Thread generator = new Thread(new FileGenerator(queue, fileCount), "generator");
28         long start = System.currentTimeMillis();
29         generator.start();
30
31         generator.join();
32         queue.close();
33         for (Thread processor : processors) {
34             processor.join();
35         }
36
37         Log.info("все файлы обработаны за " + (System.currentTimeMillis() - start) + "
38             »         мс");
39     }
40 }
```

Рисунок 26 – Класс task3.Main

3.5 Результаты выполнения

Система запускалась с генерацией 20 файлов, журнал приведён на рисунках 27 и 28.

```

C:\Users\relax\Documents\PIRKSP\Практическая работа №1>run.bat 3 20
20:52:30.254 [generator      ] создан File#1[XLS, size=88]
20:52:30.275 [processor-XLS ] начата обработка File#1[XLS, size=88] (616 мс), в очереди: 0
20:52:30.893 [processor-XLS ] обработан File#1[XLS, size=88]
20:52:31.150 [generator      ] создан File#2[XML, size=93]
20:52:31.151 [processor-XML ] начата обработка File#2[XML, size=93] (651 мс), в очереди: 0
20:52:31.288 [generator      ] создан File#3[XLS, size=33]
20:52:31.289 [processor-XLS ] начата обработка File#3[XLS, size=33] (231 мс), в очереди: 0
20:52:31.521 [processor-XLS ] обработан File#3[XLS, size=33]
20:52:31.805 [processor-XML ] обработан File#2[XML, size=93]
20:52:31.874 [generator      ] создан File#4[XML, size=56]
20:52:31.875 [processor-XML ] начата обработка File#4[XML, size=56] (392 мс), в очереди: 0
20:52:32.269 [processor-XML ] обработан File#4[XML, size=56]
20:52:32.429 [generator      ] создан File#5[JSON, size=41]
20:52:32.430 [processor-JSON] начата обработка File#5[JSON, size=41] (287 мс), в очереди: 0
20:52:32.717 [processor-JSON] обработан File#5[JSON, size=41]
20:52:33.148 [generator      ] создан File#6[XLS, size=13]
20:52:33.149 [processor-XLS ] начата обработка File#6[XLS, size=13] (91 мс), в очереди: 0
20:52:33.241 [processor-XLS ] обработан File#6[XLS, size=13]
20:52:33.811 [generator      ] создан File#7[XML, size=47]
20:52:33.812 [processor-XML ] начата обработка File#7[XML, size=47] (329 мс), в очереди: 0
20:52:34.142 [processor-XML ] обработан File#7[XML, size=47]
20:52:34.446 [generator      ] создан File#8[XLS, size=90]
20:52:34.447 [processor-XLS ] начата обработка File#8[XLS, size=90] (630 мс), в очереди: 0
20:52:35.079 [processor-XLS ] обработан File#8[XLS, size=90]
20:52:35.286 [generator      ] создан File#9[XML, size=41]
20:52:35.287 [processor-XML ] начата обработка File#9[XML, size=41] (287 мс), в очереди: 0
20:52:35.545 [generator      ] создан File#10[XLS, size=20]
20:52:35.546 [processor-XLS ] начата обработка File#10[XLS, size=20] (140 мс), в очереди: 0
20:52:35.576 [processor-XML ] обработан File#9[XML, size=41]
20:52:35.686 [processor-XLS ] обработан File#10[XLS, size=20]
20:52:36.308 [generator      ] создан File#11[JSON, size=81]
20:52:36.309 [processor-JSON] начата обработка File#11[JSON, size=81] (567 мс), в очереди: 0
20:52:36.792 [generator      ] создан File#12[JSON, size=53]

```

Рисунок 27 – Журнал работы системы обработки файлов, 20 файлов (часть 1)

```

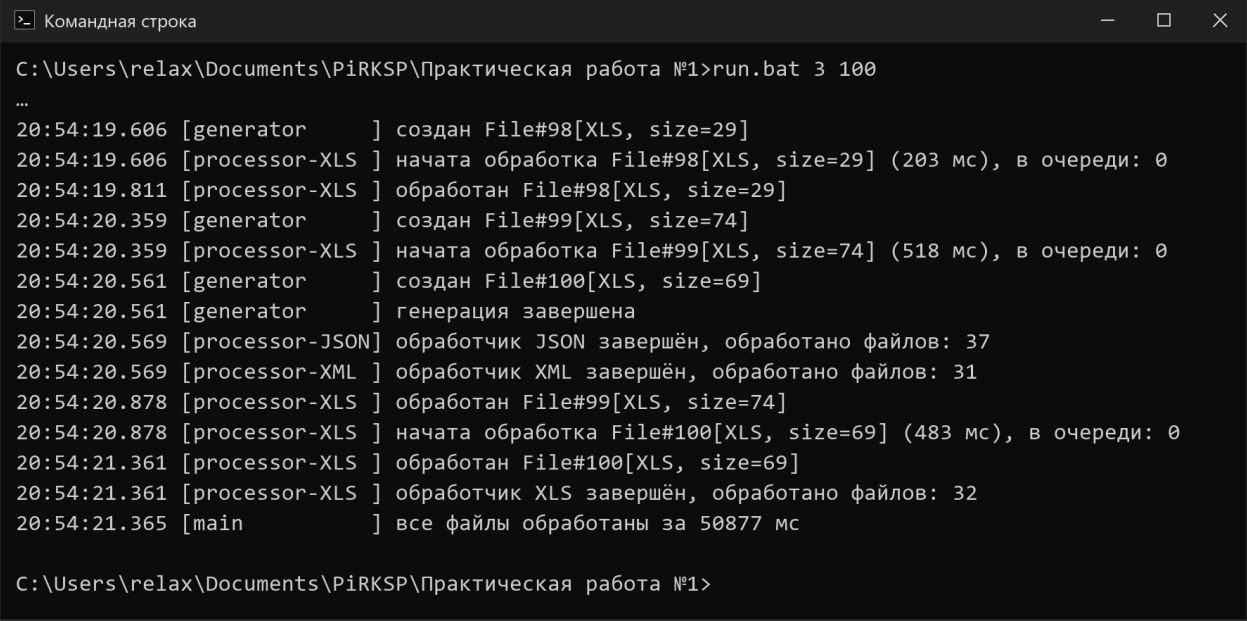
20:52:36.879 [processor-JSON] обработан File#11[JSON, size=81]
20:52:36.879 [processor-JSON] начата обработка File#12[JSON, size=53] (371 мс), в очереди: 0
20:52:37.251 [processor-JSON] обработан File#12[JSON, size=53]
20:52:37.411 [generator      ] создан File#13[JSON, size=23]
20:52:37.412 [processor-JSON] начата обработка File#13[JSON, size=23] (161 мс), в очереди: 0
20:52:37.573 [processor-JSON] обработан File#13[JSON, size=23]
20:52:38.223 [generator      ] создан File#14[XML, size=96]
20:52:38.224 [processor-XML ] начата обработка File#14[XML, size=96] (672 мс), в очереди: 0
20:52:38.897 [processor-XML ] обработан File#14[XML, size=96]
20:52:38.967 [generator      ] создан File#15[XML, size=60]
20:52:38.968 [processor-XML ] начата обработка File#15[XML, size=60] (420 мс), в очереди: 0
20:52:39.389 [processor-XML ] обработан File#15[XML, size=60]
20:52:39.627 [generator      ] создан File#16[XLS, size=75]
20:52:39.627 [processor-XLS ] начата обработка File#16[XLS, size=75] (525 мс), в очереди: 0
20:52:39.851 [generator      ] создан File#17[XML, size=26]
20:52:39.852 [processor-XML ] начата обработка File#17[XML, size=26] (182 мс), в очереди: 0
20:52:40.035 [processor-XML ] обработан File#17[XML, size=26]
20:52:40.153 [processor-XLS ] обработан File#16[XLS, size=75]
20:52:40.293 [generator      ] создан File#18[XML, size=23]
20:52:40.294 [processor-XML ] начата обработка File#18[XML, size=23] (161 мс), в очереди: 0
20:52:40.457 [processor-XML ] обработан File#18[XML, size=23]
20:52:40.633 [generator      ] создан File#19[XLS, size=11]
20:52:40.634 [processor-XLS ] начата обработка File#19[XLS, size=11] (77 мс), в очереди: 0
20:52:40.711 [processor-XLS ] обработан File#19[XLS, size=11]
20:52:41.292 [generator      ] создан File#20[JSON, size=15]
20:52:41.292 [generator      ] генерация завершена
20:52:41.292 [processor-JSON] начата обработка File#20[JSON, size=15] (105 мс), в очереди: 0
20:52:41.299 [processor-XML ] обработчик XML завершён, обработано файлов: 8
20:52:41.299 [processor-XLS ] обработчик XLS завершён, обработано файлов: 7
20:52:41.398 [processor-JSON] обработан File#20[JSON, size=15]
20:52:41.399 [processor-JSON] обработчик JSON завершён, обработано файлов: 5
20:52:41.405 [main          ] все файлы обработаны за 12027 мс

```

Рисунок 28 – Журнал работы системы обработки файлов, 20 файлов (часть 2)

Журнал показывает, что файлы разных типов обрабатываются одновременно: например, File#3 (XLS) начал обрабатываться, пока ещё обрабатывался File#2 (XML). Каждый обработчик брал только свои файлы (XML – 8, JSON – 5, XLS – 7), все 20 файлов обработаны за 12,0 с.

Для проверки на более длительном интервале система запускалась со 100 файлами; завершение журнала показано на рисунке 29.



```
Командная строка
C:\Users\relax\Documents\PiRKSP\Практическая работа №1>run.bat 3 100
...
20:54:19.606 [generator      ] создан File#98[XLS, size=29]
20:54:19.606 [processor-XLS ] начата обработка File#98[XLS, size=29] (203 мс), в очереди: 0
20:54:19.811 [processor-XLS ] обработан File#98[XLS, size=29]
20:54:20.359 [generator      ] создан File#99[XLS, size=74]
20:54:20.359 [processor-XLS ] начата обработка File#99[XLS, size=74] (518 мс), в очереди: 0
20:54:20.561 [generator      ] создан File#100[XLS, size=69]
20:54:20.561 [generator      ] генерация завершена
20:54:20.569 [processor-JSON] обработчик JSON завершён, обработано файлов: 37
20:54:20.569 [processor-XML ] обработчик XML завершён, обработано файлов: 31
20:54:20.878 [processor-XLS ] обработан File#99[XLS, size=74]
20:54:20.878 [processor-XLS ] начата обработка File#100[XLS, size=69] (483 мс), в очереди: 0
20:54:21.361 [processor-XLS ] обработан File#100[XLS, size=69]
20:54:21.361 [processor-XLS ] обработчик XLS завершён, обработано файлов: 32
20:54:21.365 [main          ] все файлы обработаны за 50877 мс

C:\Users\relax\Documents\PiRKSP\Практическая работа №1>
```

Рисунок 29 – Завершающая часть журнала работы системы, 100 файлов

Все 100 файлов обработаны за 50,9 с, после чего обработчики и генератор завершили работу. Очередь при этом не заполнялась: обработка файла длится в среднем 385 мс, а каждый обработчик получает лишь треть файлов, создаваемых в среднем раз в 550 мс.

ЗАКЛЮЧЕНИЕ

В ходе работы развиты навыки многопоточного и параллельного программирования в Java, все задачи решены.

В задании 1 обработка массива реализована последовательно, с помощью Thread и с помощью ForkJoin. Результаты всех способов совпали, а параллельные варианты оказались быстрее последовательного в среднем в 16,8 (Thread) и 13,7 (ForkJoin) раза. При равных независимых частях ручное разбиение на Thread эффективнее, а ForkJoin удобнее для рекурсивных задач.

В задании 2 с помощью ExecutorService и Future создана асинхронная программа: запросы принимаются без ожидания, результаты и исключения приходят из пула потоков в порядке готовности.

В задании 3 реализована система «производитель – очередь – потребители» с собственной ограниченной очередью на ReentrantLock и Condition. Файлы разных типов обрабатываются параллельно, файлы не теряются, а потоки завершаются штатно.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гетц, Б. Java Concurrency на практике / Б. Гетц, Т. Пайерлс, Д. Блох [и др.]. – Санкт-Петербург : Питер, 2020. – 464 с. – (Для профессионалов).
2. Lesson: Concurrency // The Java Tutorials / Oracle. – URL: <https://docs.oracle.com/javase/tutorial/essential/concurrency/> (дата обращения: 13.09.2026).
3. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления : межгосударственный стандарт : дата введения 2018-07-01. – Москва : Стандартинформ, 2017.
4. Class Thread // Java SE 21 & JDK 21 API Specification / Oracle. – URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html> (дата обращения: 13.09.2026).
5. Class ForkJoinPool // Java SE 21 & JDK 21 API Specification / Oracle. – URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ForkJoinPool.html> (дата обращения: 13.09.2026).
6. Amdahl, G. M. Validity of the single processor approach to achieving large scale computing capabilities / G. M. Amdahl // AFIPS Spring Joint Computer Conference : proceedings. – New York : ACM, 1967. – P. 483–485.
7. Interface ExecutorService // Java SE 21 & JDK 21 API Specification / Oracle. – URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ExecutorService.html> (дата обращения: 13.09.2026).
8. Interface Future<V> // Java SE 21 & JDK 21 API Specification / Oracle. – URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/Future.html> (дата обращения: 13.09.2026).